

인플루언서 서치 v2 — 설계 · PRD

influencer-agent · 2026-09-29 · v2.5 (코드 기준)

승인 · 구축 중

이 페이지가 사람이 보는 정본이다. 무엇을 · 왜 · 어떻게 동작하는지, 결정과 미결은 여기에 적는다. 프로젝트 문서 **인플루언서 서치 설계 · PRD** 는 같은 내용의 텍스트판이고(세션이 읽는 용도), 작업 방법 · 다음 할 일은 **influencer-agent** 세션 인계, 승인 대기 설계는 **조건 설계 에이전트 v4 - 제안** 에 있다.

0929 반영 — ① 결과 화면의 조건마다 **근거 게시물 링크** ② 웹 결과가 이 사람 것인지 **귀속 등급**으로 가림(D9) ③ 재발굴은 **검색 낱말을 넓히고** 진전이 없으면 멈춤(D10) ④ 버전은 **동작 지문**으로 판단(D8) ⑤ 조건 누락 막기(빠진 구절 재질의 · 원문 카드) ⑥ 평가 9의 표시 정리(①② 코드 보장 · ⑥ 결정률) ⑦ ‘뜨고 있는’ 같은 말은 코드가 **성장**(숫자 흐름)으로 바로잡음 ⑧ **정확도 측정 — 골든셋 · 버전 비교**(12장 · **eval** · **baseline** · **golden-candidates** 구현, 실모델 첫 측정 전) ⑨ 문서 정리 — 기능 요구사항 표(2~9장과 겹침) · 에이전트 계약 · LangSmith 연동 표 · 화면 목업 · 코드 구조 · 시연 대본 장은 겹치거나 옛 내용이라 뺐다. 목표 · 사용자 이야기 · 성공 지표는 1장에, 유형 16종은 4장에 남겼다.

01 한눈에

관리자 요청 (예)

“홈카페 인스타 인플루언서 중 팔로워 1만~20만, 최근 커피 브랜드 협업 안 했고 제품 단점도 말하는 사람 10명”

요청을 조건 카드로 바꿔 승인받고 → 인스타 · 유튜브에서 후보를 발굴하고 → 한 명씩 조사 · 판정해 → 조건마다 근거 링크가 붙은 결과 N명을 돌려준다.

위치

프로젝트 전체(셀럽 리서치 → 캠페인 기획 → DM → 응답) 중 리서치 단계다. 캠페인 기획 · DM 작성 · 품질 검토 · 응답 처리 위커는 자리만 있다(꺼짐).

원칙

사람이 통제할 수 있는 복잡도. 다음 할 일은 코드가 정하고(D1), LLM은 위커 안에서만 판단한다. LLM이 낸 값(핸들 · 링크 · 계정 · 판정)은 코드가 틀 결과와 대조한다.

스택

Python · LangGraph · FastAPI(Railway) / Next.js(Vercel) / Supabase **agentops** (운영 기록) / LangSmith(트레이스) / OpenAI(기본 **OPENAI_MODEL**, 조건 생성은 **PLANNER_MODEL**)

목표

- 자연어 요청 1회로 조건에 맞는 인플루언서를 요청 인원만큼(기본 10명) 받고, 인물마다 인스타 · 유튜브 · 웹 정보를 조건별 근거와 함께 한 화면에서 본다.

- 2. 정해진 필터가 아니라 어떤 자연어 조건이든 가진 도구로 잴 수 있는 방법으로 바뀌 승인받는다. 조건을 빠뜨리지 않고, 잴 수 없는 조건은 실행 전에 알린다.
- 3. 새 업무(캠페인 기획 · DM · 응답 대응)를 기존 코드를 고치지 않고 워커 추가로 붙인다.
- 4. 어떤 버전이든 배포 · 디버그 · 개선 판정을 AgentOps가 근거와 함께 내리고, 그 판정 없이 운영에 배포하지 않는다(배포 연동은 아직).

사용자 이야기

#	누가	무엇을	상태
U1	관리자	요청 한 줄을 넣고 조건 카드(원문 구절 · 측정 방법 · 기준값 · 가능성 · 기준표)를 보고 승인 · 기준 수정 · 필수↔참고 · 빼기 · 말로 고치기를 한다	됨
U2	관리자	결과 여러 명을 비교하고, 조건마다 근거 게시물 · 다른 플랫폼 계정을 찾은 경로와 같은 사람일 확률 · 조건별 확인률을 본다	됨
U3	관리자	인물마다 👍/👎와 이유를 남긴다 → 평가 데이터 · 골든셋 후 보가 된다	됨
U4	운영자	검색 하나를 열어 총괄 → 워커 → LLM · 톨 호출을 시간순으로, 탈락 · 오류는 이유와 함께 본다	됨
U5	운영자	프롬프트 · 에이전트를 바꾼 뒤 같은 골든셋으로 다시 채점해 이전 버전과 비교하고 배포 판정을 받는다	일부 판정 화면만 — 골든 20 · 비교 리포트 · 판정 기록은 아직
U6	개발자	새 에이전트를 shadow로 붙여 운영 결과에 섞지 않고 점수를 모은 뒤, 판정이 DEPLOY면 켜다	아직

성공 지표 — 배포 판정 기준과 같다

품질 종합 점수 ≥ 0.80 (점수 평가 4개) · 치명 평가 4개 전부 통과 · 기준 대비 하락 ≤ 0.03 · 검색 p95 $\leq 240s$ (10명 기준, 사람 대기 제외) · 비용 $\leq \$0.02 \times$ 요청 인원(최소 \$0.20) · 오류율 $\leq 2\%$ · 트레이스 누락 0 · 승인된 조건의 측정식 검증 통과 100% · 가능성 예측과 실제 확인률 일치 $\geq 70\%$ · 조건 판정 pass 정밀도(골든 20 판정 라벨 — 기준은 첫 측정 뒤 정한다).

범위 밖: 메신저 연동 · 실제 DM 발송(워커 자리만) · 다중 사용자 권한 · 벡터DB.

진행

단계	상태
1 탐색 · 설계 · 2 화면 목업 · 3 틀 셋업	완료
4 구축	진행 중 남은 것: 조건 설계 v4(승인 대기) · O7 · 게이트 기록/배포 연동 · 커밋 · 배포
5 기능 검증	골든셋으로 정확도 첫 측정 (12장)
6 LLMOps 진단 시연	게이트가 IMPROVE → 수정 → DEPLOY로 바뀌는 한 바퀴

02 사용 흐름 (화면)

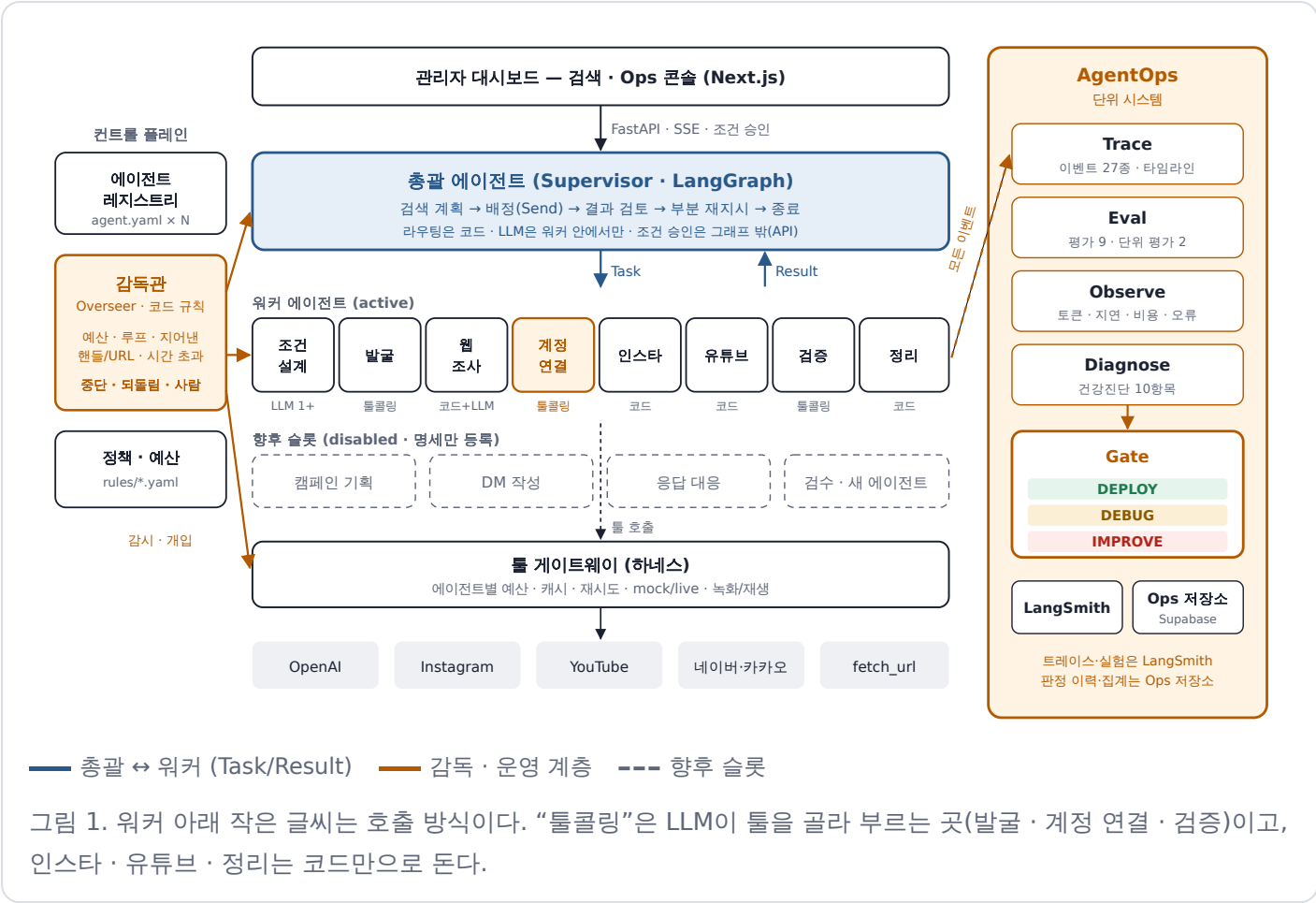
1. **요청** → **조건 카드**: 카드마다 원문 구절 · 유형 · 필수/참고 · 기준표 · 판정 방법, 예상 비용. 말로 고치기(“참여율은 빼줘”) · 카드 수정 후 승인.
2. **진행**: 단계(계획 · 발굴 · 웹 조사 · 계정 연결 · 유튜브 · 인스타 · 판정 · 정리) + 로그(라운드별 새 검색어 · 재조사 · 재발굴 · 멈춘 이유 · 감독관 개입).
3. **결과**: 제목 줄(인원 · 시간 · 비용, 모자라면 “요청 N명 중 M명 · K라운드 · 검색어 J개로 다시 찾았고 · 멈춘 이유 · 빠진 이유”) · 조건별 확인률 · 결과 표(CSV) · **팔로워 확인 필요**(인스타 조회가 안 된 계정 — 찾은 글과 함께) · 제외된 후보(한 사람씩 사유).
4. **상세 패널**: 요약 · 조건(판정 · 근거 게시물 1 · 2 · 3 — 조건을 보여 준 게시물 · 영상 주소, 계정 단위 조건만 ‘프로필 보기 (게시물 근거 없음)’ · 인스타그램 · 유튜브(찾은 경로 · 같은 사람일 확률 · ‘확인 필요 · 판정에 안 씀’) · 웹(본인 확인 먼저, ‘이름만 일치 · 동명이인 주의’ 배지) · 👍/👎 + 이유.
5. **Ops 콘솔**(/ops) — 9장 · 정확도는 12장.

브루다이어리 @brew.diary 통과 예시

팔로워	4.8만 ✓ 1만~20만 · 프로필 보기 (게시물 근거 없음)
다른 계정	유튜브 @brewdiary · 같은 사람일 확률 0.9 (서로 링크)
협찬 이력	✓ 충족 최근 90일 커피 협찬 표시 0건
단점도 말함	✓ 충족 근거 게시물 1 · 2 · 3 — “산미가 좀 아쉬웠어요” 등

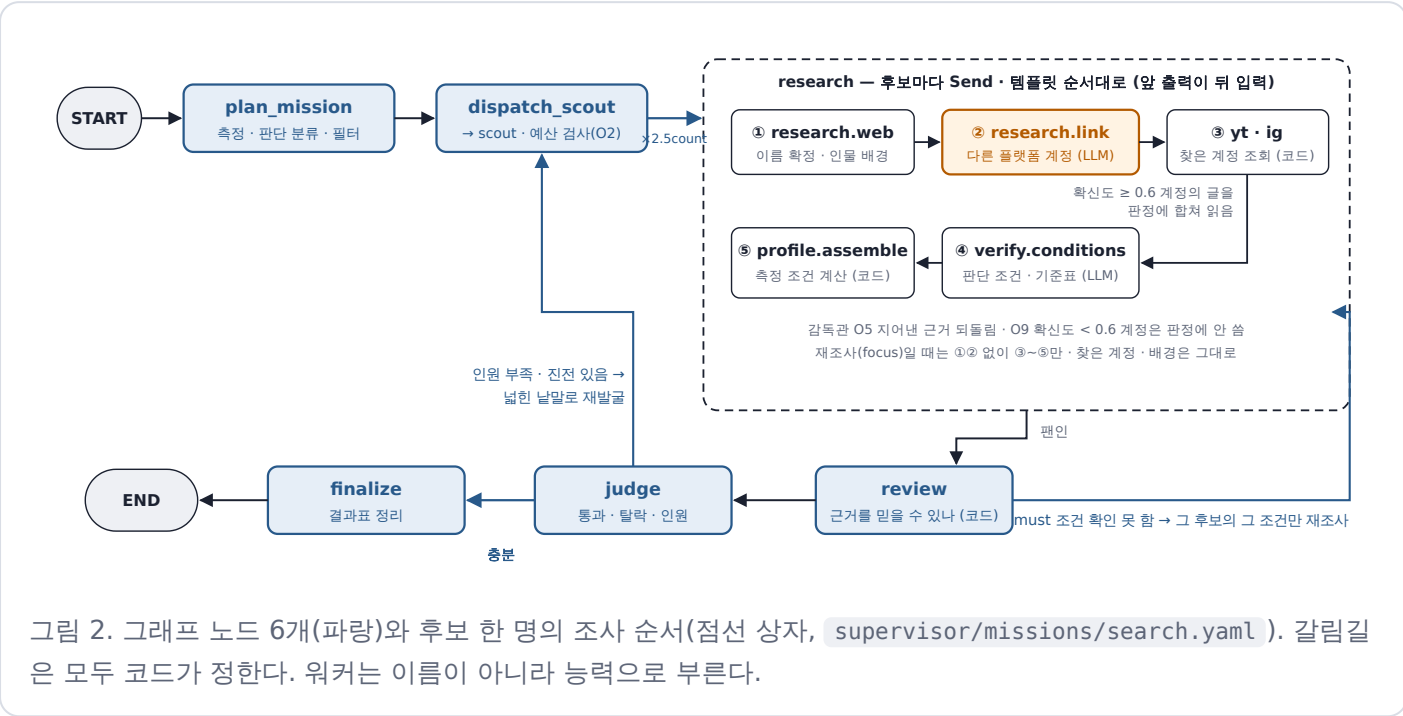
03 구조 — 총괄 · 워커 · 감독관

총괄은 검색을 쪼개고 결과를 검토한다. 워커는 한 가지 일만 한다. 감독관은 일을 하지 않고 규칙을 어기면 멈추거나 되돌린다. AgentOps는 이 모두를 기록하고 판정한다.

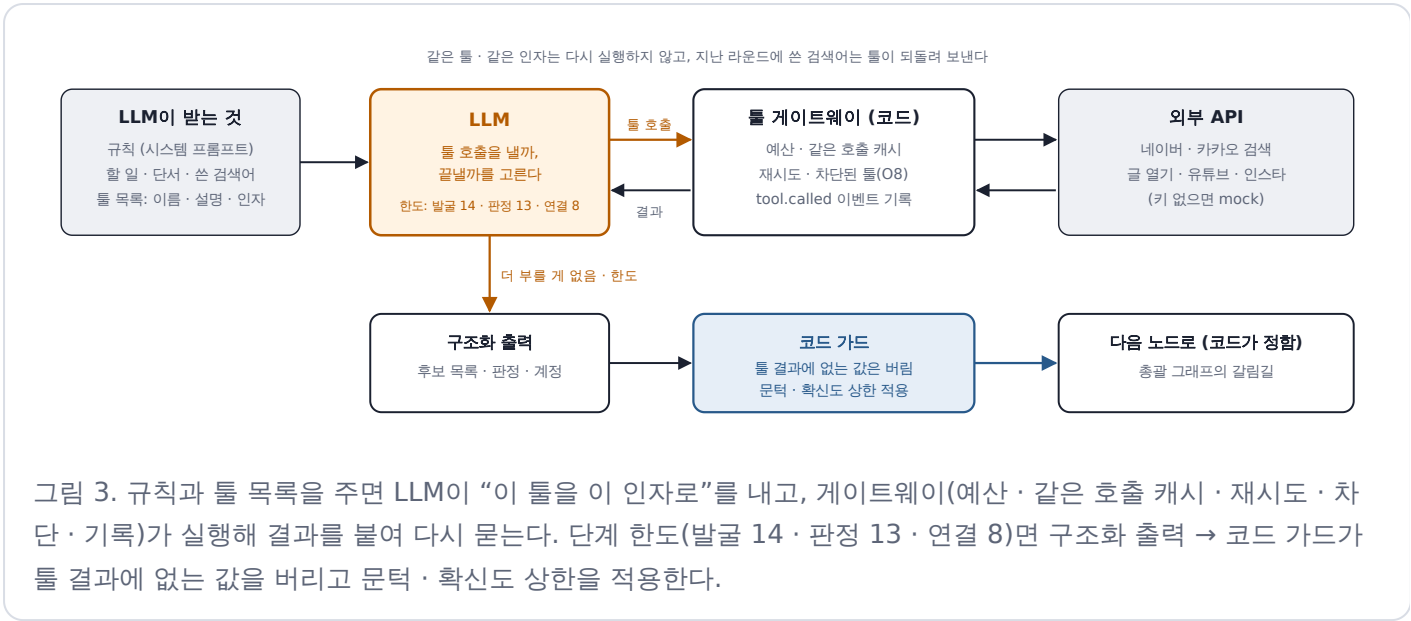


워커	하는 일	누가 판단
query-planner	요청문 → 조건 카드 (그래프 밖 · 승인 전)	LLM(조건 생성 모델) + 코드
scout	발굴 → 숫자 필터 → 주제 선별 · 재발굴 때 검색어 넓히기	코드가 먼저 소개글 계정 모으기, 그다음 LLM 검색
web-researcher	이름 확정 · 인물 배경 · 결과 귀속	코드(고정 절차) · LLM은 뽑아 적기만
account-linker	다른 플랫폼 계정 찾기	LLM(툴 선택) + 코드 가드
yt- · ig-researcher	채널 · 프로필 · 지표	코드
verifier	증거 조건 판정	LLM(툴 선택) + 코드 문턱
profiler	측정 조건 계산 · 카드	코드
꺼집 campaign-planner · dm-writer · qc · reply-handler	다음 단계 자리	—

그래프와 후보 한 명의 조사 순서



LLM이 툴을 고르는 곳 — 발굴 · 계정 연결 · 판정 셋뿐



LangGraph는 어떻게 쓰이나

그래프는 하나다(`agent/supervisor/graph.py` 의 `StateGraph(MissionState)`). 에이전트마다 그래프를 따로 만들지 않는다 — 워커는 그래프 노드가 아니라 `research` 노드 안에서 순서대로 부르는 파이썬 함수다.

LangGraph 기능	여기서 쓰는 곳
노드 6개	<code>plan_mission</code> · <code>dispatch_scout</code> · <code>research</code> · <code>review</code> · <code>judge</code> · <code>finalize</code>
고정 엣지	<code>START</code> → <code>plan_mission</code> → <code>dispatch_scout</code> · <code>research</code> → <code>review</code> · <code>finalize</code> → <code>END</code>
조건부 엣지 (코드 함수)	<code>dispatch_scout</code> → <code>research</code> 또는 <code>review</code> (후보 0명) · <code>review</code> → <code>research</code> (근거 보강) 또는 <code>judge</code> · <code>judge</code> → <code>dispatch_scout</code> (재발굴) 또는 <code>finalize</code>
<code>Send</code> (병렬 팬아웃)	후보마다 <code>Send("research", ...)</code> — 후보 N명을 동시에 조사. 재조사는 그 후보 · 그 조건만(focus)
상태 (MissionState)	조건 · 후보 · 조사 결과 · 라운드 · 쓴 검색어 · 멈춘 이유. 병렬 결과는 리듀서로 합친다
안 쓰는 것	체크포인트 · <code>interrupt</code> (조건 승인은 그래프 밖 API에서 먼저) · 서브그래프 · LLM 라우팅

LLM 호출은 그래프가 아니라 워커 안에서 일어난다: 조건 생성(그래프 밖) · 발굴 · 계정 연결 · 판정은 툴 루프(`llm.run_tool_agent`), 주제 선별 · 검색 낱말 넓히기는 1회 호출. 호출은 모두 LangSmith 트레이스와 `agentops.events` 에 남는다.

감독관 (코드 규칙 · LLM 없음)

규칙	감지	개입
O1 예산	작업의 LLM · 톨 한도 초과	작업 중단 → partial
O2 검색 예산	비용 > 예산 × 1.5 또는 유튜브 유닛 > max(900, 50 × 인원)	새 배정 중지 → 있는 결과로 종료
O3 루프	같은 배정 3회째	거부 → unknown 확정
O4 지어낸 핸들	톨 결과 어디에도 없는 핸들	후보 폐기
O5 지어낸 근거	근거 URL이 톨 결과에 없음 · 채널 홈을 근거로 씀	그 판정을 unknown으로
O6 시간 초과	작업 마감 초과	취소 → partial
O7 모델 오류 연속 미구현	같은 모델 오류 3회 연속	대체 모델
O8 외부 차단	토큰 만료 · 쿼터 · 공급사 401/403	그 톨을 검색 끝까지 차단 · 다음 조건 카드에 경고
O9 동일인물 불확실	계정 연결 확신도 < 0.6	보여 주되 ‘확인 필요 · 판정에 안 씀’

한 검색이 도는 모습 — 위 홈카페 10명

0

조건 카드

LLM

코드

query-planner

분야 · 규모(발굴 필터) · 협찬(없어야 함) · 성향(기준표). 성향 구절이 빠지면 코드가 그 구절만 다시 묻는다(4장).

1

발굴

코드

LLM이 톨 고름

scout

코드가 find_instagram_accounts("홈카페 인스타 계정 추천") → 소개글 속 @계정 → LLM이 더 검색 → 코드가 근거 없는 핸들 · 사이트 이름을 버림 → 인스타 조회 → 팔로워 필터 · 주제 선별.

웹 조사

귀속

코드

"브루다이어리" 인스타그램어 인터
뷰 · "brew.diary" → 귀속 등
급으로 거름

계정 연결

유튜브 찾기

LLM

search_mentions → 닉네임이라
판단 → find_youtube_channel
→
lookup_youtube("@brewdiary")
→ 설명란 "Instagram:
brew.diary" → 0.9

지표

유튜브 · 인스타

코드

게시물 · 영상 · 참여율

판정

증거 조건

LLM

성향은 모은 글로, 협찬은
web_search 1번 → 문턱은 코드

정리

측정 조건

코드

규모 · 협찬 표시 수 계산 · 카드

근거가 부실한 조건만 재조사 → 2/10이면 재발굴(넓힌 낱말 '라떼아트 · 핸드드립 ...') → 진전이 없으면 멈추고 이유를 결과에 적는다(5장 · D10).

04 조건 만들기 (컴파일러 v3)

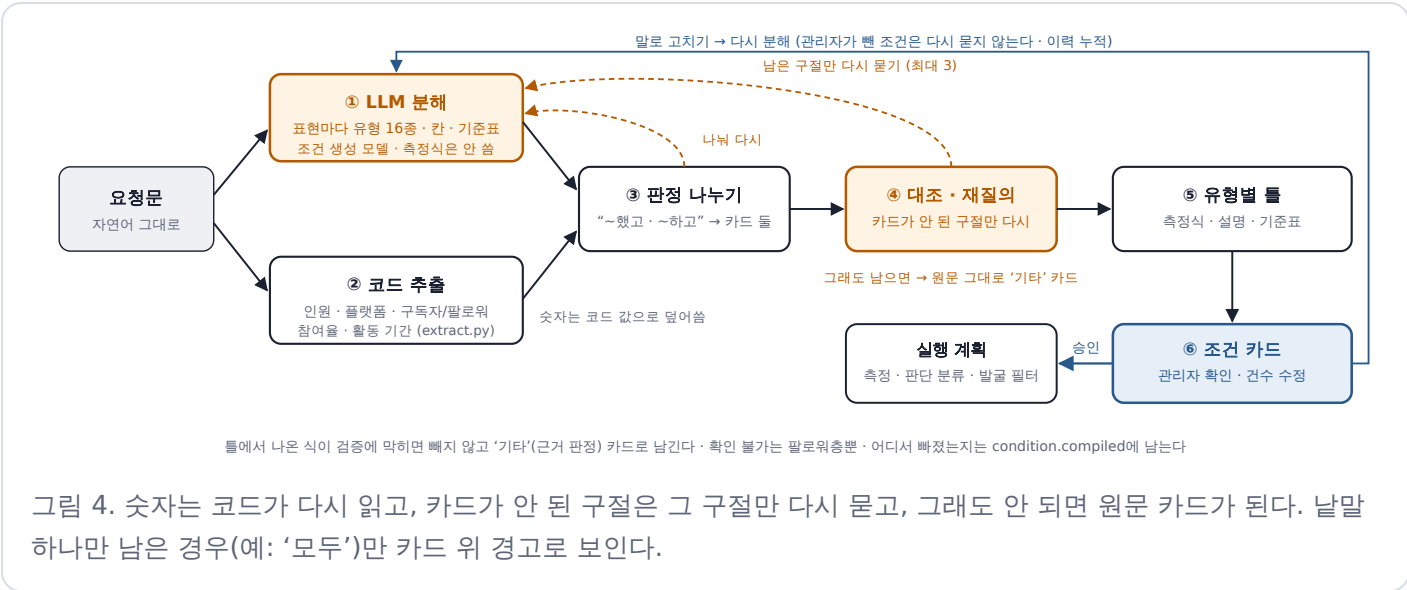


그림 4. 숫자는 코드가 다시 읽고, 카드가 안 된 구절은 그 구절만 다시 묻고, 그래도 안 되면 원문 카드가 된다. 낱말 하나만 남은 경우(예: ‘모두’)만 카드 위 경고로 보인다.

compiler.compile_request 의 순서 — LLM은 1 · 3 · 5단계만 부른다. (그림 4의 ①~⑥은 같은 흐름을 묶어 그린 것이다)

단계	누가	하는 일
1 분해	LLM 1회	규칙 · 유형표 16 · 예시 → 표현마다 원문 · 유형 · 칸 · 기준표 · 발굴 검색 각도
2 원문 대조	코드	원문이 요청문에 없는 조건은 뺀다 — 지어낸 조건 막기
3 판정 나누기	구절당 1회	‘~했고 · 하고 · 않고 · 이며 · 면서’로 판정 둘이 붙은 카드는 나눈다(카드 하나 = 판정 하나)
4 숫자 · 플랫폼	코드	인원 · 범위 · 참여율 · 활동 기간 · ‘둘 다’를 요청문에서 읽어 덮는다
5 남은 구절 되살리기	코드 + 구절당 1회	어느 카드에도 안 쓰인 낱말이 든 구절만 다시 묻고(최대 3), 그래도 남으면 원문 그대로 ‘기타’ 카드. 이어서 성장 바로잡기(코드 · 0929): ‘뜨고 있는 · 떠오르는 · 라이징 · 급상승 · 요즘 핫한 · 주목받는’ 구절이 성향 · 기타 · 평판 · 사실(기준표 — 캡션에서 그 낱말 찾기)로 분류됐으면 ‘성장’으로 바꾼다. 기간은 구절의 ‘N개월 · 주 · 일’(없으면 90일)
6 유형별 틀	코드	측정식 · 기준표 · 대안 · 발굴 필터. 분야 게이트(c0)와 브랜드 · 매장 공식 계정 제외(cb)는 코드가 항상 붙인다. 10개 넘게 잘리면 메모
7 카드 · 기록	코드	예상 비용 · 막힌 틀 경고, condition.compiled 에 LLM 원출력 · 뺀 것 · 재질의 · 잘림

유형 16종 — LLM이 고르는 것

유형	예	재는 방법 (코드 틀)
규모 · 참여율 · 활동 · 인원 · 플랫폼	팔로워 1만~20만 · 참여율 3% · 최근 3개월 활동 · 10명 · 둘 다	코드가 요청문에서 직접 읽는다 — “숫자 그대로” 카드 + 발굴 필터
분야	IT 리뷰 · 패션 · 홈카페	<code>topic_gate</code> — 코드가 항상 붙인다
콘텐츠	스마트폰 비교 영상을 올리는	주제 낱말 × 형식 낱말이 맞은 게시물 · 영상 수(1년 안 1건 이상) — 맞은 게시물 링크가 근거
협찬	협찬 사례 있는 · 최근 커피 브랜드 협업 안 한	협찬 표시(#광고 · #협찬 · #제공 · 유료 광고 · ‘협업’) × 브랜드 범주 낱말을 센다. ‘최근’만 있으면 90일, 0건이면 충족. 대안: 표시 없는 협업을 캡션 · 기사로
성장	요즘 뜨고 있는 · 라이징	글이 아니라 숫자 흐름으로 — ① 유튜브 최근 N일 조회수 중앙값이 이전 N일의 1.3배 이상 ② 인스타 최근 N일 반응(좋아요+댓글) 중앙값이 1.3배 이상 ③ 최근 N일 업로드 수가 이전보다 많음 ④ 최근 N일 기사 · 정리글에 ‘라이징 · 주목’ 언급 — 4개 중 2개(대체 지표 · 배수와 개수는 카드에서 수정). 실제 팔로워 증가율은 API가 과거 수치를 주지 않아 못 잴다
지역언어	국내 · 일본에서 활동하는	한글 비율, 그 밖은 추정
인물	여성 · 20~30대 · 현직 개발자	호칭 · 소개 · 생활 단계로 추정 — 속성마다 카드 하나
성향	제품 단점도 말하는 · 내돈내산 위주 · 친근한 말투	기준표 + 판정이 글을 읽어 판단, 문턱은 코드
사실 · 평판	삼성과 협업한 · 논란 없는	기준표 + 기사 · 본인 콘텐츠 근거
팔로워층	20대 여성 팔로워가 많은	확인 불가(API 없음) — 대안: 그 대상을 겨냥한 콘텐츠로 추정
기타	위에 안 맞는 모든 것 · 5단계 원문 카드	버리지 않는다 — 기준표를 붙여 근거로 판정

- **말로 고치기**: 1단계부터 다시. 관리자가 뺀 카드는 되살리지 않고, 지시와 무관하게 빠진 카드는 되살린다.
- **성장 바로잡기 계기**(0929): 패션 요청에서 작은 모델(nano)이 ‘뜨고 있는’을 기준표로 만들어 충족 신호가 ‘뜨고 있는 · 인기 있는’이 됐다 — 본인 캡션에 그 낱말이 있어야 충족이라는 뜻이 된다.

- **기준표**(성향 · 사실 · 평판 · 기타): 충족 기준 한 문장 · 충족/반대 신호 · 읽을 곳 · 기간 · min_hits(카드에서 수정) · min_read.
- **조건 생성 모델**: `PLANNER_MODEL` (없으면 `LLM_HIGH_MODEL` , 기본 모델과 같으면 안 씀 · 실패하면 기본 모델). 맥 `.env` 는 둘이 같아 **`PLANNER_MODEL=gpt-4.1-mini`**를 넣어야 켜진다(검색당 약 +\$0.01).

칸	기준표 예 — “제품 단점도 말하는”
기준	제품 · 원두 · 도구를 소개하는 게시물에서 아쉬운 점이나 단점을 직접 적는다
충족 신호	아쉬운 점 · 단점은 · 호불호 · 별로였 · 비추 · 장단점
반대 신호	모든 제품을 인생템 · 강추로만 소개 · 광고 문구만 있는 캡션
판정 (코드)	신호가 보인 게시물 2건 이상 → 충족 · 5건 이상 읽고 0건 → 미충족

한계 — 같은 홈카페 요청 6번 중 5번 ‘제품 단점도 말하는’이 빠졌고(프롬프트 예시에 같은 문장이 있는데도), ‘뜨고 있는’을 기준표로 만들었다(작은 모델의 분류 실수). 5단계 코드 가드로 막지만, 근본 해결은 `조건 설계 에이전트 v4` (누락 = 형식 실패 · 판정 전략 5 · 조건별 판정 — 11장 1).

05 발굴 · 재발굴 (scout)

- 인스타는 **API**로 주제 발굴이 안 된다(7장). 그래서 소개글 속 계정을 모은다:
`find_instagram_accounts` = 검색 → 추천 · 정리 글 4개 열기(네이버 블로그는 모바일 주소) → 본문
의 `@id` · `instagram.com/id` · 인스타: `id` 를 문맥과 함께. 유튜브는 `youtube_search` (채널ID가 확
실).
- 가드: 툴 결과에 계정 표기로 적힌 핸들만 받는다(지어낸 핸들 · 사이트 이름 `@namu.wiki` · 한글 핸들
· 도메인 버림). 툴 없이 답한 후보는 전부 버린다.
- 선별: 후보당 API 1회(병렬) → 숫자 필터 → 주제 선별(LLM 1회). 인스타 조회 불가('Invalid user id'
— 개인 계정 · 없는 계정 구분 불가) + 팔로워 조건 → 탈락 대신 '팔로워 확인 필요'.
- 갈때기(D7): 후보 8 × 인원(20~240) → 조사 2.5 × 인원(6~75).
- 참여율: 구간 수치(`rules/engagement.yaml`)는 참고값 — 탈락에 쓰지 않는다. 관리자가 숫자를 줄
때만 거른다. 플랫폼 간 비교 안 함.

재발굴 — 인원이 모자랄 때

장치	내용
검색 낱말 넓 히기 (LLM 1 회)	분야의 하위 주제 · 동의어 · 연관 분야(홈카페 → 홈바리스타 · 라떼아트 · 핸드드립 · 모카포 트 ...). 조건 낱말(협업 · 협찬 · 광고 · 단점 · 팔로워 ...)은 코드가 뻥다 — '협업 안 한' 요청에 '협업'을 검색하면 정반대가 나온다. 쓴 낱말은 다시 쓰지 않는다
코드 검색량	라운드마다 소개글 계정 모으기를 2 + 모자란 인원/3번(최대 6) , 넓은 낱말 먼저, 4개씩 동 시에. 새 낱말은 LLM 발굴 지시에도 넣는다(유튜브는 LLM이 <code>youtube_search</code> 로)
처방	지난 라운드 사유별 처방 · 쓴 검색어는 툴이 거부 · 새 후보 0명 라운드 뒤엔 검색 방식 전환
멈춤 (D10)	채움 · 두 라운드 연속 새 후보 0명 · 한도(4라운드)를 넘으면 지난 라운드에 새로 조사까지 간 후보가 있을 때만 계속 · 한도의 2배(8) · O2 예산. 멈춤 이유는 결과 화면에 보인다

실측 계기(0928): 홈카페 10명 요청이 4라운드 모두 돌아갔지만 새 후보 18 · 2 · 5 · 3명(조사까지 5 · 1
· 1 · 1명)으로 2명에서 멈췄다. 검색어는 같은 말을 조금씩 바꾼 것뿐이었다.

6-1. 웹 조사와 귀속 — “이 사람 이야기인가” (D9)

- **이름 확정:** 표시 이름이 사람 이름 모양이면 그대로(신뢰도 0.9), 아이디뿐이면 아이디로 검색해 이름과 아이디가 **같은 글에 함께** 있는지 본다. 신뢰도 0.5 미만이면 웹 검색 결과를 이 사람 것으로 보지 않는다(계정 안 콘텐츠만 근거).
- **검색:** "<이름>" 유튜버/인스타그램어 인터뷰 · <이름> 나무위키 · "<아이디>" (협찬 후기 · 인터뷰는 아이디를 태그한 글에 가장 확실하다). 검색어의 '@'는 떼고(카카오에서 '@' 검색 83%가 0건), 메일 주소는 검색어로 쓰지 않는다.

귀속 등급 (코드)	조건	쓰임
본인 확인	앵커가 글에 있음 — 아이디 · 채널ID · 설명란에 본인이 적은 인스타 · 본인 주소 · 소개란 메일의 개인 도메인 · 확신도 0.6 이상 연결 계정	근거로 씀
이름 일치	이름이 독립된 낱말(‘이탈리아’는 ‘리아’가 아니다) + 크리에이터 문맥이나 분야 낱말. 짧은 이름(한글 2자 · 영문 3자 이하)은 둘 다	‘이름만 일치 · 동명이인 주의’ 배지
무관	그 밖	버림

- 나무위키도 예외 없이 **본문까지** 이 사람 이야기여야 읽는다. 위키는 단서일 뿐 근거가 아니다.
- ‘이 사람에 대해’는 걸러 낸 결과에 있던 주소의 사실만 남긴다(코드 보장). 판정 LLM에게 주는 web_search도 후보 전용이라 무관한 결과는 보이지 않는다.
- 대가: 짧거나 흔한 이름이면서 아이디 · 문맥이 없는 글은 놓친다.

6-2. 다른 플랫폼 계정 (account-linker)

LLM이 검색어 · 톨을 고르고, 잘라낸 글을 읽어 아이디/닉네임을 가린 뒤 API로 확인한다. **이름이 달라도 받는다** — 같은 사람인지는 내용으로 본다(강: 상호 링크 · 한 글에 두 계정 · 본인 설명란 / 보통: 같은 소재 · 행사 · 협업 / 약: 아이디 · 이름 비슷함만).

톨	주는 것	비용
search_mentions	검색 → 언급 구절 · 글에 적힌 계정 표기 · 귀속 등급	무료
read_passages	본문에서 언급 주변 5구절	무료
lookup_instagram	이름 · 소개 · 웹사이트 · 팔로워 · 캡션 5	토큰 필요
lookup_youtube	채널 · 소개 · 구독자 · 제목 5 · 설명란 인스타	3유닛
find_youtube_channel	닉네임 → 채널 후보 (후보당 1회)	100유닛

- 한도: 톨 합쳐 12회 · LLM 8단계. 가드: 톨 결과에 없는 아이디 · 닉네임 모양 인스타 id 버림 · LLM이 조회 안 했으면 코드가 조회(유튜브 조회 실패는 버림) · 인스타 토큰 만료면 확신도 상한 0.7, 그 밖 조회 실패 0.5 · 플랫폼당 가장 유력한 1개.

- O9: 0.6 미만은 '확인 필요 · 판정에 안 씀'. 0.6 이상 계정의 게시물 · 영상은 판정의 '읽을 글'에 [인스타] · [유튜브] 로 합친다.
- 한계: 확신도는 LLM이 매긴다 — 이름 검색만으로 찾은 채널이 0.9로 붙은 사례가 있다(→ 11장 2).

6-3. 판정 (verifier · judge)

- verifier는 증거 조건만 판정한다(측정 조건은 profiler). 읽을 글(캡션 · 설명란)에 코드가 기준표 신호 낱말을 ★로 표시하고, LLM은 뜻으로 판단한다(표시가 있어도 뜻이 다르면 신호가 아니다).
- 근거는 항목 단위 링크(영상 · 게시물 · 기사) — 채널 홈 · 프로필 루트 · 위키만 달린 pass는 unknown으로 되돌린다. 행동 조건은 채널 안 검색(youtube_channel_search)을 먼저.
- 결과 화면의 근거: 판정마다 links (근거 게시물 · 영상 최대 5개)와 source_kind (post · profile). 기준표 조건은 신호가 보인 게시물 전부, 측정 조건은 조건에 걸린 게시물 — 충족이면 맞은 글, 미충족이면 어긋난 글(예: '협찬 안 한'인데 #광고 글). 계정 · 채널 주소는 게시물 근거가 없을 때만 '프로필 보기 (게시물 근거 없음)'로 보인다. 인스타 게시물 주소는 Graph API의 permalink다.
- 추정 조건(성별 · 나이대 · 생활 단계): 본인 지칭 · 소개 · 말투 신호 여럿이 모이면(70%+) pass. 사진 속 얼굴로 판단하지 않는다.
- 배제 조건: 위반 근거를 찾았을 때만 fail. '없음'은 증명할 수 없으므로 링크 없는 fail은 unknown.
- 문턱은 코드(enforce_rubric): pass인데 신호 링크 < min_hits → unknown · unknown인데 min_read 이상 읽고 0건 → fail · 읽은 게 모자란 fail → unknown · 목록 · 톨 결과에 없는 링크는 인정하지 않는다(O5).
- review: 근거가 부실한 후보는 그 조건만 재조사(최대 2회). judge: 필수 fail · 배제 위반 → 탈락. 필수 unknown은 탈락이 아니라 '근거 약함' — 정원이 모자라면 뒤에 채운다(배지). 순위 = 참고 조건 충족 수 → 근거 없는 조건이 적은 순 → 참여율.

07 외부 API · 제약

API	제약 · 대응
웹 검색	네이버 → 카카오 → Tavily · Serper · Brave → Google. 401/402/403은 그 공급사를 빼고 기록 · 화면 경고. 네이버 · 다음은 <code>site:</code> 를 따르지 않는다. 운영 검색 451건이 전부 카카오였다 — Railway에 네이버 키 필요 (네이버 블로그의 '@계정' 소개글이 한국 인스타 발굴의 주 출처)
인스타 Graph API (0913 실측)	해시태그 → 게시물은 되지만 작성자는 주지 않는다. business_discovery는 username을 이미 알아야 하고 프로 · 크리에이터 계정만. 없는 계정과 개인 계정 모두 'Invalid user id'. 운영은 장기 토큰(60일 · 시스템 사용자). 발굴 통과율이 유튜브보다 낮아(유튜브 30명 → 2~8명, 인스타 18명 → 0~4명) 후보 수가 사실상 유일한 레버다
유튜브 Data API	search.list 100유닛(발굴 · 채널 찾기 · 채널 안 검색) · 채널 조회 3유닛 · 하루 10,000유닛

08 예산 · 비용

- 톨 호출 한도는 인원예 비례(`rules/budget.yaml`), 80%면 경고, 넘으면 호출하지 않고 LLM에게 알린다(O1). 검색 전체는 O2.
- 실측: 패션 30명 **\$0.084**(gpt-4.1-nano · 토큰 77%가 판정) · 홈카페 10명 4라운드 \$0.067 · 420초 · IT 리뷰 10명 3라운드에서 유튜브 902유닛으로 O2 중지. 재발굴이 8라운드까지 가면 시간이 대략 두 배. 계정 연결 추정 +\$0.004/명.

09 AgentOps · Ops 콘솔

- 이벤트 27종 → Supabase `agentops.events` (+ 로컬 jsonl). 봉투에 검색 · 작업 · 에이전트 · 동작 지문.
- 버전 (D8 · 이유는 10장): 모든 이벤트의 `prompt_version` · `runs.meta.build` · `scores.build` 에 동작 지문(`build_id`)이 실리고, '같은 버전인가'는 지문으로 판단한다. 진단 H4(평가 신선도)도 지문이 같은 평가만 '이 버전'으로 인정한다 — 지문 없는 예전 점수는 깨끗한 커밋 SHA가 정확히 같을 때만(unknown · -dirty 는 인정 안 함).
- 화면: 개요(배포 판정 · 막힌 것 · 검색 추이) · 검색 추적(문제 · 오류 왜 · 탈락 왜 · 후보 격자 · 비용 · 시간 · 톨) · 관측(SLO) · 품질 평가 · 사람 평가 · 골든셋 · 에이전트 · 진단 · 배포 판정. 한 곳에서 쟀다 (`agentops/measure.py` → `/api/ops/measure`, 지난 24시간 · 3건 미만이면 최근 20건 · 모의 제외).
- 관측 SLO 10: 지연 p95(인원 대비) · 첫 결과까지 · 토큰 · 예산 초과 비율 · 오류율(검색/작업) · 재시도 · 유튜브 유닛 · 캐시 · 부분 결과 + 에이전트 p95 · 모델별 · 공급사 분포 + 단계별 품질(발굴 · 계정 연결 · 확인률 예측 일치(성공 지표 $\geq 70\%$ · $\pm 20\%$ p) · 비용 추정 정확도).
- 기록: `condition.compiled` (LLM 원출력 · 뺀 것 · 재질의 · 원문 카드) · `condition.coverage` (`scope` = 판정까지 간 후보 전원 · 탈락자 포함, `returned` = 결과 인원 — 결과 화면의 확인률은 결과 기준 그대로, Ops 기록만 전원 기준. 결과만 세면 fail 난 사람이 이미 빠져 fail이 늘 0이었다: 0928 실측 27행) · `web.attributed` (후보마다 본인 확인 · 이름 일치 · 버림 · 잘린 사실) · scout 검토의 새 검색어 · judge 검토의 멈춘 이유.
- 사람 평가: 🍌/🍌 + 이유(다른 사람 · 조건 판정 틀림 · 캠페인에 안 맞음 · 정보 틀림 · 기타) → 대리 지표 · 추이 · 골든셋 후보(12장). 탈락 · 오류: 한 사람씩 단계 · 사유 · 세부 근거 / 오류 원인 규칙 16 가지 → 제목 · 왜 · 할 일.

평가 9	어떻게 재나	주의
치명 ① 지어낸 계정 0 · ② 근거 없는 pass 0	코드 보장 — 가드가 결과 전에 버린다	결과엔 늘 0건이라 측정값이 아니다. 막은 건수를 보이고 반복은 H9
치명 ③ 요청에 없는 조건 0	조건 분해 평가 no_extra (<code>eval</code> 48건)	—
치명 ④ 남의 정보 섞임 0	사람 평가 '다른 사람' 대리 지표	결과에 오른 사람만 평가받아 선택 편향
⑤ 인원 충족 · ⑧ 🍌 비율	운영 기록	—
⑥ 결정률	판정을 내린 비율	정확도가 아니다 — 종합에서 뺌
⑦ 골든 20 정답 일치	골든 요청 20 (12장 R3)	아직 없음
⑨ 주제 적합도	사람 평가 '캠페인에 안 맞음' 대리 지표	선택 편향

종합 = 점수 평가(⑤⑦⑧⑨) 중 쟀 것의 평균. 단위 평가 점수는 `agentops.scores` 에 버전 · 지문과 함께 저장한다. 진단 H1~H10(키 · API · 쿼터 · 평가 신선도(지문 기준) · 회귀 · SLO 소진 · 오류 군집 · 드리프트 · 감독관 개입률 · 무결성).

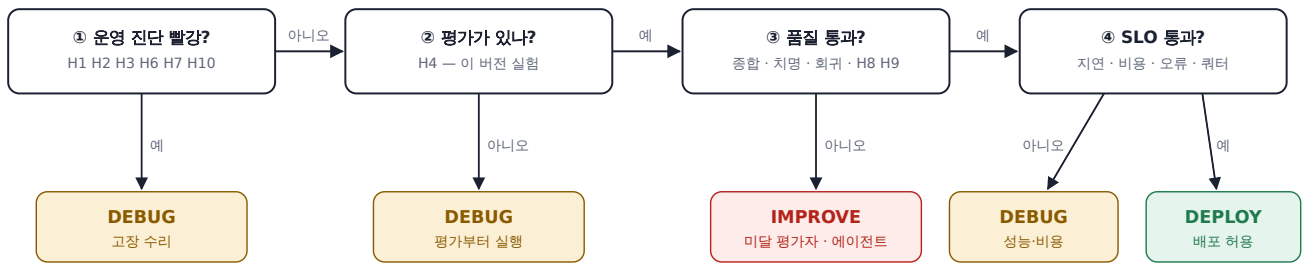


그림 5. 게이트의 판정 순서. 운영이 고장 난 상태의 점수는 믿을 수 없으므로 품질보다 먼저 본다. 지금은 계산 · 표시만 하고 기록 · 자동 배포는 없다. 골든 요청이 없어 ②에서 늘 DEBUG다.

10 결정

#	결정
D1	총괄 = LangGraph + 코드 라우팅. LLM은 워커 안에서만
D2	판정(verify)을 소스별 조사와 분리
D3	감독관은 코드 규칙 (LLM 아님)
D4 · D5	LangSmith(무료) + Supabase <code>agentops</code>
D6	조건 컴파일러 — LLM은 유형 · 칸, 측정식은 코드 · 빠진 구절은 코드가 되살림 · 숫자 흐름으로 짚 말(‘쓰고 있는’)은 코드가 ‘성장’으로 바로잡음
D7	갈때기 · 예산은 인원 비례
D8	‘같은 버전’은 커밋 SHA 가 아니라 동작 지문으로 — 배포 이미지엔 .git이 없어 SHA가 unknown이 고(0928 실측: 이벤트 2,572건 중 2,214건), 커밋 전 로컬 실행은 옛 SHA로 찍혀 SHA로는 버전을 못 가린다. <code>build_id()</code> = agent · rules 파일 내용(코드 · 프롬프트 · 규칙) + <code>OPENAI_MODEL</code> · <code>PLANNER_MODEL</code> 의 해시 8자리. <code>git_sha()</code> 는 사람이 읽는 용도로 두고 커밋 안 된 변경이면 - <code>dirty</code> , Dockerfile은 Railway 커밋 SHA를 <code>GIT_SHA</code> 로 받는다. 구현 Supabase <code>scores.build</code> 마이그레이션 적용 · 커밋 전
D9	웹 결과가 이 사람 것인지는 이름이 아니라 앵커로 코드가 가린다
D10	재발굴은 라운드 수가 아니라 진전(새로 조사할 후보)과 예산으로 멈춘다 (한도 2배는 안전장치)
D11 구현	정확도는 정답이 하나인 일만 골든셋으로, 바뀐 에이전트만, 바뀌었을 때 잰다 (12장)

11 미결 · 제안 (승인 필요)

- 1 조건 설계 에이전트 v4 — 별도 문서. 누락 = 형식 실패 · 유형표 16 → 판정 전략 5 + 증거 카탈로그 · 조건별 판정 · 파도식.
- 2 계정 연결 확신도 상한을 코드가 — 한쪽 소개 · 설명란이 다른 쪽을 가리킴 0.95 · 한 글에 두 계정 0.8 · 이름 · 아이디 비슷함만 0.5('확인 필요'). 근거: @deena_mom → 이름 검색만으로 찾은 유튜브 채널이 0.9. 프로토타입에서 이 사례는 0.5, 설명란으로 찾은 계정은 0.95 유지.
- 3 본인 블로그(선택) — 소개란 · 링크모음 · 설명란의 `blog.naver.com/<id>` 를 앵커로 → 공개 RSS 최근 글 10개를 '읽을 글'에 [블로그] 로. 코드 전용 · 무료(RSS 접근은 배포 후 확인).
- 4 정확도 측정 프로세스 — 12장 전체(골든셋 R1~R4 · 언제 재나 · 버전 비교 규칙 · 만들 것 순서). 12-6의 1 · 2 · 4 구현 완료 — 남은 것은 사람 작업(첫 측정 · 30쌍).
- 5 비용 절감안 — 측정 조건 먼저 거르기 · 틀 결과 줄이기 · 새 단서가 있을 때만 재조사 · 파도식 판정 · 설명란 단서가 있으면 계정 연결 단축(앞의 둘 · 파도식은 v4에 포함).
- 6 발굴 남은 문제 — 검색 각도에 조건 낱말이 섞인다(LLM 발굴 지시에는 아직 그대로) · 인스타 조회 불가로 빠지는 후보가 많다(0명으로 끝난 검색은 4라운드 모두 조사 0명) · 발굴 LLM이 글을 열지(`fetch_url`) 않는다 · 분야가 다른 계정이 후보로 오른다(토익 강사 계정이 홈카페 후보로) → 선별에서 분야 확인 강화.
- 7 운영 설정 — `PLANNER_MODEL` · Railway 네이버 키 · 장기 인스타 토큰 · 배포.

12

정확도 측정 — 골든셋 · 버전 비교

구현 0929

골든셋은 사람이 확인한 정답 모음이다. 에이전트가 낸 답을 이 정답과 맞춰 봐야 정확도를 숫자로 말할 수 있다. 한 번 확정된 정답은 버전이 바뀌어도 재사용한다 — 새 버전이 나오면 같은 문제를 다시 풀게 하고 같은 정답으로 채점해 이전 버전과 비교한다. 사람이 버전마다 다시 확인하지 않는다.

지금은 정확도를 재지 못한다 — 조건 골든셋은 유형만 채점하고, 계정 연결 정답은 0쌍, 판정 정답은 없다(화면의 품질 숫자는 대부분 가드 산출값 · 결정률 · 👍 대리 지표). 아래 요구사항 중 1 · 2 · 4와 3의 화면은 구현됐다(커밋 전). 남은 것은 실모델 첫 측정(=기준점)과 계정 정답 30쌍 확정이다.

12-1. 무엇을 어떻게 재나 — 에이전트별

에이전트	방법	이유	지금
query-planner (조건)	골든셋 — 요청문 → 기 대 카드	정답이 하나로 정해진다	48건 · 방식 · 반복 채점 구현 · 실모델 첫 측정 전
account-linker (같은 계정)	골든셋 — 계정 → 다른 플랫폼 정답(또는 ‘없 음’)	정답이 하나로 정해진다	화면 · 저장 구현 · 확정 0쌍
verifier (판정)	골든셋(사람 라벨) — 나중	라벨 비용이 크 다 · 위 둘 다음	없음 배포 판정 ②가 늘 DEBUG인 이유
scout (발굴)	운영 지표(채움률 · 주 제 적합 · 빈 라운드)	‘정답 명단’이 없는 열린 검색	젠다
web-researcher (귀속)	운영 지표 + 상세 패널 ‘본인 맞음/아님’ 표본	—	운영 지표만
yt · ig- researcher · profiler	단위 테스트(pytest · LLM 0)	코드라서 평가 가 아니라 테스 트	테스트

12-2. 골든셋 요구사항

#	골든셋 · 한 문제	채점 · 목표
R1	조건 <code>agentops/eval/decompose_cases.yaml</code> (48건 · 목표 60 · 확 인용 14) — 요청문 → 기대 카드: 유형 · 측정 방식 (숫자 흐름 · 글 읽기 · 코드 숫자) · 숫자 · 기간 · 부정 여부. 실제로 틀렸던 요청(패 션 ‘뜨고 있는’ · 홈카페 ‘제품 단점도 말하는’)과 같은 뜻 다른 말 (‘요즘 핫한’ · ‘급성장’)을 먼저 넣는다	표현 재현율 $\geq 95\%$ · 완전 일치 $\geq 80\%$ · 지어낸 조건 0 · 충족 신 호가 요청문 낱말 그대로면 감점. 같은 요청 3회 — 3번 다 맞아야 정답(작은 모델은 매번 다르다)
R2	계정 연결 <code>agentops.golden_links</code> · <code>/ops/golden</code> (30쌍 이상) — (발굴 플랫폼 · 계정) → (다른 플랫폼 · 정답 계정 또는 ‘없음’)	정밀도 $\geq 95\%$ · 오연결 0(치명) · 재현율 $\geq 70\%$ · ‘없음’ 맞힘 · 확신도 구간별 정밀도
R3	판정(나중) — 골든 요청 20건 실행 → 결과 인물 · 필수 조건마다 맞음/틀림 + 탈락자 표본	조건 유형별 pass 정밀도 · precision@k
R4	저장	점수는 <code>agentops.scores</code> 에 동 작 지문 · 커밋과 함께, 틀린 사례 (요청 · 기대 · 실제)는 파일로. 정 답지는 Git(yaml) 또는 Supabase에 두고 바꾼 사람 · 날 짜를 남긴다

12-3. 계정 연결 정답이 만들어지는 순서

1

검색하면서 기록 코드

account.link_result

계정 연결 에이전트가 후보마다 어느 플랫폼을 찾아야 했고 · 어떤 계정을 붙였고 · 확신도가 얼마였는
지 남긴다. 못 찾은 경우도 남아 놓친 것까지 셀 수 있다.

2

확인할 후보 모으기 코드

harvest

이 기록과 결과 카드의 사람 평가(‘다른 사람 · 계정 오인’)를 모아 확인 대기 목록에 올린다. 먼저 볼 순
서: 사람이 ‘다른 사람’이라고 한 것 → 확신도 0.4~0.8(오판이 많은 곳) → 못 찾은 것 → 확신도 높음
(기준 보정용). ‘다른 플랫폼 없음’ 사례를 20% 이상 넣는다(없는데 붙이는 실수를 잡으려고).

3

사람이 증거를 보고 확정 사람

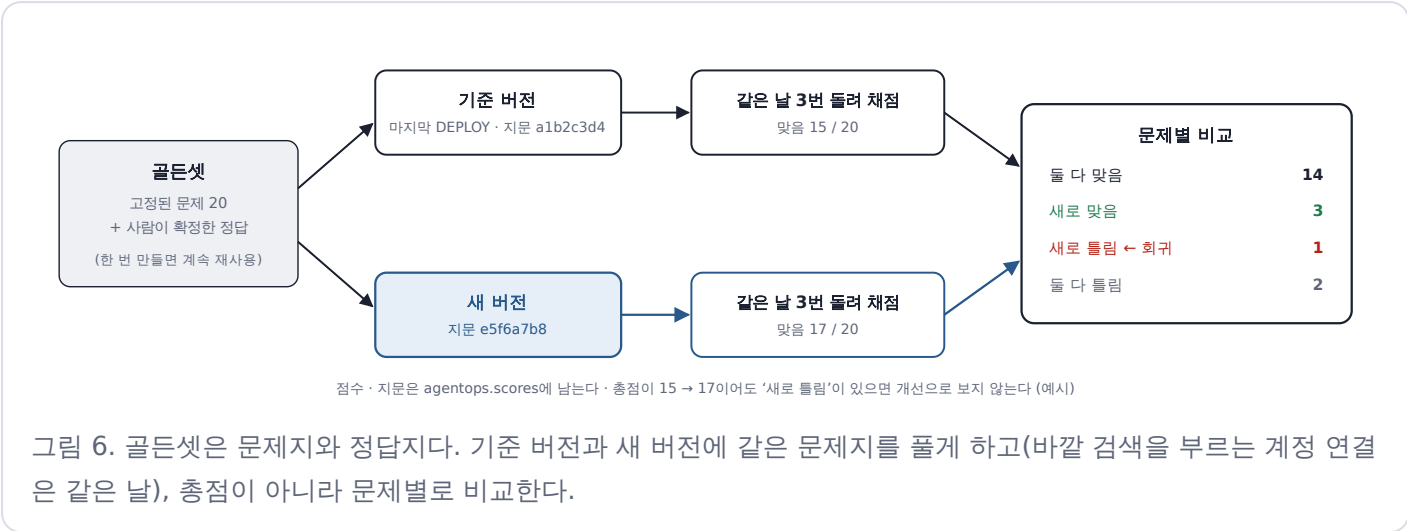
/ops/golden

맞음 / 틀림(정답 주소 입력) / 계정 없음 / 모르겠음. 증거 등급 — ① 한쪽 소개 · 설명란 · 링크모음이
다른 쪽 주소를 적음(서로 가리키면 가장 확실) ② 한 글(기사 · 블로그 · 협업 공지)에 두 계정이 함께
적힘 ③ 같은 얼굴 · 콘텐츠 · 협업이 두 곳에 올라옴(사람이 직접 봄). ①이나 ②가 있거나 ③을 사람이
확인해야 ‘맞음’이고, 이름 · 아이디가 비슷할 뿐이면 ‘모르겠음’(골든셋에서 뺀다). ‘계정 없음’은 소개
란 · 링크모음 · 설명란에 다른 플랫폼이 없고 이름 · 아이디 검색에도 본인 계정이 안 나올 때. 확정할
때 증거 링크 1개를 남긴다(칸은 만들 것 3). 처음 10쌍은 관리자가 직접 아는 크리에이터로 채워도 된
다. 한 번 확인한 것은 후보를 다시 모아도 덮어쓰지 않는다.

불인 것 중 맞은 비율(정밀도 · 기준 $\geq 95\%$) · 찾아야 할 것 중 찾은 비율(재현율 $\geq 70\%$) · 다른 사람을 불인 경우(오연결 · 치명) · ‘없음’ 맞힘 · 확신도 구간별 정밀도 → O9 기준(0.6)을 올릴지 내릴지 제안. 30쌍 이상 모아서 본다.

12-4. 언제 재나 · 새 버전이 더 정확한가

- **바뀌었을 때만:** 그 에이전트의 프롬프트 · 모델 · 조건 규칙 · 코드 가드를 바꿨을 때, 배포 전에 그 에이전트의 골든셋만. 동작 지문이 같으면 다시 재지 않는다. 비용: 조건 60건 × 3회 약 \$0.15 · 계정 30쌍 약 \$0.15.
- **배포 판정:** 이 지문으로 돌린 점수가 없으면 DEBUG(H4), 기준 버전보다 떨어지면 IMPROVE(H5).
- **평소 운영:** 골든셋은 돌리지 않고 운영 신호만 본다(👉와 이유 · 감독관 개입 · 조건 누락 · 성장 바로 잡기 메모 · 확신도 낮은 연결). 주 1회 또는 운영 검색 50건마다 운영에서 나온 실패를 골든셋 후보로 모아 사람이 확정한다(30분).



	운영 기록 대조	다시 돌리기 (eval-link · eval-decompose)
무엇을 채점하나	그때 그 버전이 실제 검색에서 불인 결과	지금 버전에게 확정된 문제를 다시 풀게 한 결과
버전 비교에 쓰나	참고만 — 버전마다 조사한 사람이 달라 공정한 비교가 아니다	기준 — 같은 문제 · 같은 정답
어디에	골든셋 화면의 버전별 정확도	agentops.scores (버전 · 동작 지문과 함께)

“더 좋다”를 판단하는 규칙

1. **기준 버전과 비교한다.** 기준은 마지막으로 배포 판정을 통과한 버전. 새 점수가 기준보다 0.03 넘게 떨어지지 않고, 치명(오연결 · 지어낸 조건) 0.
2. **문제별로 본다.** 총점이 올라도 기준이 맞힌 문제를 새 버전이 틀렸으면(새로 틀림) 퇴보가 섞인 것이다. ‘새로 틀림’이 비어 있어야 개선이라고 말한다.
3. **같은 문제로만 비교한다.** 골든셋이 늘었으면 공통 문제로만 비교하거나 기준 버전도 새 문제로 다시 돌린다.

4. **3번 돌린다.** LLM은 같은 입력에도 답이 달라진다 — 조건은 3번 다 맞아야 정답(R1), 계정 연결은 평균. 20문제면 한 문제가 5%p라 1~2문제 차이는 '비슷하다'로 보고 문제 수를 늘린다.
5. **바깥 세상이 바뀌는 걸 막는다.** 조건 골든셋은 입력이 요청문뿐이라 기준 점수를 그대로 쓴다. 계정 연결은 실제 웹 검색 · API를 불러 검색 결과가 바뀌면 에이전트가 같아도 점수가 달라진다 — 톨 응답 녹화 · 재생이 없는 동안은 **비교할 때 기준 버전도 같은 날 다시 돌린다.**
6. **골든셋에만 맞추지 않는다.** 틀린 문제를 보고 프롬프트를 고치면 그 문제에만 맞춰질 수 있다. 정답의 1/3은 고칠 때 보지 않는 확인용으로 떼어 두고, 운영에서 🙌 받은 사례를 계속 새 문제로 더한다.

12-5. 재고 반영하는 흐름

1. **바꾼다** — 프롬프트 · 모델 · 코드 가드 중 하나만(무엇이 효과였는지 알 수 있게).
2. **젠다** — `python -m agentops eval` (조건 3회 반복 + 계정 정답이 있으면 함께). 출력: 요청마다 정확 N/N · 빠짐 · 더 나옴 · 재는 방식 틀림 · 요청 낱말을 신호로 옮김 → 기준 대비 점수 ▲▼ → **새로 틀림 · 새로 맞음**. 배포 뒤 `python -m agentops baseline` 으로 그 빌드를 기준으로 등록한다.
3. **비교한다** — Ops 품질 평가 화면의 버전별 추이 · 문제별 비교. 떨어지면 배포 판정이 IMPROVE로 막는다.
4. **반영한다** — 틀린 사례를 원인별로 묶어 쓴 것부터: 조건 생성 모델(`PLANNER_MODEL`) → 틀린 유형별 코드 가드(예: 성장 바로잡기) → 프롬프트 예시 → 구조 변경(v4). 계정 연결은 확신도 구간별 정밀도로 O9 기준(0.6)을 조정하고 확신도 상한을 코드로(11장 2).
5. **쌓는다** — 운영에서 틀린 사례는 정답을 달아 골든셋에 영구히 넣는다(같은 실수가 다시 나오면 바로 잡힌다).

시스템은 **재고 · 비교하고 · 막고 · 틀린 사례를 보여 주는 데까지**만 한다. 무엇을 고칠지는 사람이 정한다(자동 수정 없음).

12-6. 만들 것 (순서)

#	할 일	크기
1 ✓	조건 골든셋 48문제 — 측정 방식(metric · rubric · evidence) 채점 · 요청 문구 베끼기 감점 · <code>--repeats 3</code> (모두 맞아야 정답) · 실제 실패 사례 d44~d48. '뜨고 있는' 류는 코드가 성장(좋아요 추이) 조건으로 바로잡음(<code>fix_growth</code>)	완료
2 ✓	<code>python -m agentops eval</code> 한 명령 · 문제별 새로 틀림/새로 맞음 · H5 회귀(기준 빌드 대비 2%p↓ 또는 오연결 증가 = 빨강) · 확인용 14문제(holdout, 기본 숨김) · <code>python -m agentops baseline</code> 으로 배포 빌드를 기준점 등록	완료
3	계정 연결 정답 30쌍 — 골든셋 화면에서 확정(증거 등급 ①②③/없음 · 증거 링크 칸 추가됨). 배포 후 검색 10~20건 → 관리자 확정	1~2시간(사람)
4 ✓	<code>python -m agentops golden-candidates</code> — 🙌 조건 틀림 · 코드가 바로잡은 조건을 <code>candidates.yaml</code> 초안으로(정답은 사람이 채움)	완료
5	(나중) 판정 골든셋 R3 · LLM 판정 평가자(사람 라벨 100개 이상 · TPR/TNR ≥ 0.85 로 검증한 뒤) · 신뢰구간 · 톨 응답 녹화 · 재생	—

13 운영 가이드 — 관리자가 하는 일

시스템이 재고 · 비교하고 · 막는다. 사람은 정답을 확정하고, 무엇을 고칠지 정하고, 배포를 누른다.

13-1. 주기별 할 일

언제	할 일	어디서	시간
검색할 때마다	결과 카드에 👍/👎 + 이유(특히 ‘다른 사람’ · ‘조건 판정 틀림’)	검색 결과 · 상세 패널	몇 초
매일 1번	개요의 배포 판정 · 막힌 것(토큰 만료 · 쿼터 · 오류 군집) 확인	/ops 개요 · 진단	5분
주 1회 또는 검색 50건마다	① 계정 연결 후보를 증거 보고 확정(맞음 · 틀림 · 계정 없음 · 모르겠음 + 증거 등급 · 링크) ② golden-candidates 로 모은 조건 후보에 정답을 채워 decompose_cases.yaml 에 추가	/ops/golden · yaml	30 분
프롬프트 · 모델 · 규칙 · 가드를 바꿨을 때	13-2 버전 올리기	터미널	10 분

13-2. 버전 올리기 (바꾸기 → 재기 → 배포)

- 1

하나만 바꾼다

사람

프롬프트 · 모델(OPENAI_MODEL · PLANNER_MODEL) · rules/*.yaml · 코드 가드 중 하나. agent/ · rules/ 파일이나 모델 이름이 바뀌면 동작 지문(build_id)이 자동으로 바뀐다 — 버전 번호를 손으로 매기지 않는다.
- 2

짚다

코드

python -m agentops eval

조건 골든셋 48문제 × 3회(+ 계정 정답이 있으면 계정 연결). 약 \$0.15 · 몇 분. 출력 끝의 기준 대비 ▲▼와 새로 틀림을 본다.
- 3

판단한다

사람

새로 틀림 0 · 지어낸 조건 0 · 기준 대비 하락 ≤ 2%p면 통과. 아니면 틀린 사례를 보고 다시 1로. 확인용 14문제 점수(holdout)가 개발용보다 크게 낮으면 골든셋에 맞춰 고친 것이다.
- 4

커밋 · 배포

사람

git push → Railway · Vercel

배포 판정이 DEPLOY인지 /ops 배포 판정에서 확인(지금은 표시만 — 자동 배포 없음).

배포한 빌드를 다음 비교의 기준으로 남긴다. 이후 H5 회귀는 이 빌드와 비교한다.

13-3. 명령 모음 (백엔드 폴더에서)

<code>python -m agentops eval</code>	조건 골든셋 3회 + 계정 연결 · 기준 대비 · 새로 틀림/맞음
<code>eval-decompose --ids d44,d45 --repeats 3</code>	일부 문제만
<code>eval --show-holdout</code>	확인용 14문제 상세까지 보기(고칠 때는 보지 않는다)
<code>eval-link</code>	계정 연결 골든셋만
<code>golden-candidates</code>	👉 · 코드가 바로잡은 조건 → <code>agentops/eval/candidates.yaml</code> 초안
<code>baseline</code>	지금 빌드를 기준으로 등록(배포 직후)

13-4. 계정 연결 정답 확정 기준 (요약)

맞음 = ① 한쪽 소개 · 설명란 · 링크모음이 다른 쪽을 적음 · ② 한 글에 두 계정이 함께 · ③ 같은 콘텐츠 · 협업을 사람이 직접 확인 중 하나. **계정 없음** = 소개란 · 링크모음 · 설명란에 없고 이름 · 아이디 검색에도 안 나옴. 이름만 비슷하면 **모르겠음**(골든셋에서 빠짐). 목표 30쌍 · 그중 '없음' 20% 이상.

13-5. 문서 관리

이 페이지(아티팩트)가 사람용 정본이고, 고칠 때마다 PDF로 다시 뽑아 프로젝트에 올린다. 프로젝트 문서는 5개 — `PRD.pdf` (이 페이지 인쇄본) · 1. PRD (무엇 · 왜 · 결정 · 미결) · 2. 설계서 (구조 · LangGraph · 조건 · 판정 · AgentOps · 승인 대기 v4) · 3. 운영 가이드 (이 13장) · 4. 세션 인계 (작업 방법 · 상태 · 다음 할 일).

근거: influencer-agent 코드(커밋 1e5dfb8 + 맥의 커밋 전 변경) —

`agent/conditions/compiler.py` · `agent/nodes/discover.py` · `verify.py` · `judge.py` ·
`agent/harness/context.py` (`git_sha` · `build_id`) · `agentops/golden.py` · `eval/link.py` ·
`measure.py` , Supabase `agentops` 실측(0927~0929 — 이벤트 2,572건 중 `git_sha unknown` 2,214 · 조건 확인률 27행 fail 전부 0 · 홈카페 요청 6번 중 5번 '제품 단점도 말하는' 누락), 인스타그램 API 실측(0913). 요금은 2026년 공개 정리 기준이다.